

UNIVERSITY OF ASIA PACIFIC
DEPARTMENT OF CSE

Lab Report - 5

COURSE CODE: CSE 402.

COURSE TITLE: OPERATING SYSTEM LAB.

SUBMITTED BY:

NAME : Morium Hossain Mim

STUDENT ID : 23101134

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C - 1

DATE : 10/12/2026

SUBMITTED TO

ATIA RAHMAN ORTHI

LECTURER AT UAP

1. Problem:

Given the following processes with their **Arrival Time (AT)**, **Burst Time (BT)**, and **Priority**:

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	3	4	2
P2	0	4	0
P3	2	3	1
P4	1	6	1
P5	4	2	3

Implement the following CPU Scheduling Algorithms:

- **Priority Non-Preemptive Scheduling**
- **Priority Preemptive Scheduling**

Assume that a **smaller priority number indicates a higher priority**.

For each algorithm, calculate the following for every process:

- **Completion Time (CT)**
- **Turnaround Time (TAT)**
- **Waiting Time (WT)**

Also, calculate:

- **Average Turnaround Time (Average TAT)**
- **Average Waiting Time (Average WT)**

Finally, display the results for all processes and **compare the Average TAT and Average WT of both Preemptive and Non-Preemptive Priority Scheduling algorithms to determine which algorithm performs better for the given processes.**

2. Source Code:

priority scheduling(preemptive)

```
process = ["P1", "P2", "P3", "P4", "P5"]
at = [3, 0, 2, 1, 4]
bt = [4, 4, 3, 6, 2]
priority = [2, 0, 1, 1, 3]

n = len(process)


ct = [0]*n
wt = [0]*n
tat = [0]*n
tq = 3
remaining = bt.copy()
time = 0
completed = 0



while completed < n:
    x = -1
    for i in range(n):
        if at[i] <= time and remaining[i] > 0:
            if x == -1 or priority[i] < priority[x] or (priority[i]
== priority[x] and at[i] < at[x]):
                x = i
    if x == -1:
        time += 1
    else:
        if remaining[x] > tq:
            time += tq
            remaining[x] -= tq
        else:
            time += remaining[x]
            remaining[x] = 0
            ct[x] = time
            completed += 1



for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]

print("Process\tAT\tBT\tCT\tTAT\tWT")
```

```
for i in range(n):

print(f"{process[i]}\t{at[i]}\t{bt[i]}\t{ct[i]}\t{tat[i]}\t{wt[i]}"
)


print("\nAverage TAT =", sum(tat)/n)
print("Average WT =", sum(wt)/n)
```

priority scheduling(non-preemptive)

```
process = ["p1", "p2", "p3", "p4"]
at = [0,3,1,2]
bt = [3,2,2,1]
priority = [4,0,2,1]

n = len(process)


ct = [0] * n
wt = [0] * n
done = [0] * n
tat = [0] * n

time = 0
completed = 0



while completed < n:

    x = -1
    for i in range(n):

        if at[i] <= time and done[i] == 0:

            if x == -1 or priority[i] < priority[x]:
                x = i

    if x == -1:
        time += 1
```

```
    else:
        time = time + bt[x]



        ct[x] = time
        done[x] = 1
        completed += 1




for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]



print("Process\tAT\tBT\tCT\tTAT\tWT")



for i in range(n):
    print(process[i], "\t", at[i], "\t", bt[i], "\t", ct[i], "\t",
tat[i], "\t", wt[i])



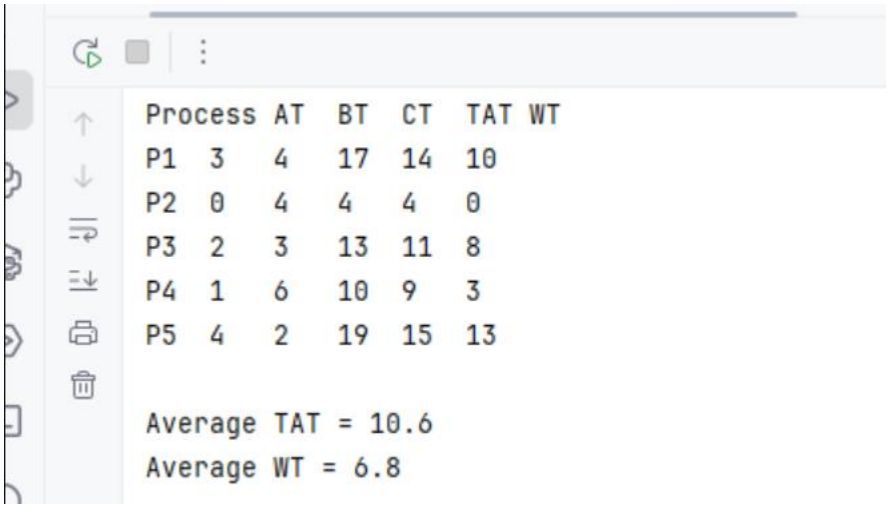
non_P_avg_tat = sum(tat)/n
non_p_avg_wt = sum(wt)/n



print("\nAverage TAT =", non_P_avg_tat)
print("Average WT =", non_p_avg_wt)
print("\n")
```

3. Output:

priority scheduling(preemptive)



```
Process AT  BT  CT  TAT WT
P1  3   4   17  14  10
P2  0   4   4   4   0
P3  2   3   13  11  8
P4  1   6   10  9   3
P5  4   2   19  15  13

Average TAT = 10.6
Average WT = 6.8
```

priority scheduling(non-preemptive)

```
C:\Users\Windows\Documents\ml
Process AT  BT  CT  TAT WT
p1  0   3   3   3   0
p2  3   2   5   2   0
p3  1   2   8   7   5
p4  2   1   6   4   3

Average TAT = 4.0
Average WT = 2.0
```

4. Discussion:

In this experiment, **Priority Non-Preemptive** and **Priority Preemptive CPU Scheduling** were implemented using the same group of processes. Their performances were compared by analyzing the **Average Turnaround Time (TAT)** and **Average Waiting Time (WT)**.

For **Priority Non-Preemptive Scheduling**, the obtained Average TAT was **4.0**, while the Average WT was **2.0** . In this method, a process that gets the CPU keeps running until it finishes, even if another process with a higher priority arrives during its execution.

For **Priority Preemptive Scheduling**, the Average TAT was **10.6** and the Average WT was **6.8** . Here, the currently running process may be stopped when a new process with a higher priority enters the ready queue.

From the experimental results, the **Non-Preemptive Priority algorithm** achieved better values for both Average TAT and Average WT. Hence, for the given set of processes, the Non-Preemptive approach provided better overall performance.

5. Conclusion:

The experiment demonstrated the implementation and performance comparison of **Preemptive and Non-Preemptive Priority Scheduling**. The results show that the **Non-Preemptive Priority algorithm** performed more efficiently for the selected processes, with an Average TAT of **4.0** and an Average WT of **2.0** .

In contrast, the **Preemptive Priority algorithm** resulted in an Average TAT of **10.6** and an Average WT of **6.8** . Since both performance measures were lower in the Non-Preemptive method, it can be considered the better choice for this particular dataset.

However, the Preemptive Priority approach has an advantage in situations where a high-priority process arrives and requires CPU service without much delay.

6. GitHub:

priority scheduling(non-preemptive)

https://github.com/23101134Mim/CSE_402_Operating_System_Lab/tree/main/priority%20scheduling(non-preemptive)

priority scheduling(preemptive)

https://github.com/23101134Mim/CSE_402_Operating_System_Lab/tree/main/priority%20scheduling(preemptive)